Simple Dictionary

Part 3 — Platforms, build setup, browser worker and tests

Phone edition • Source snapshot: 28 October 2026

How to use this document

Each source line has its own label, code and explanation. This edition includes imports, blank lines and closing brackets too.

Code is above its explanation rather than beside it. Leading indentation is removed and long code is wrapped to fit a phone. A wrapped code block is still one original source line.

Line numbers refer to the HTML walkthrough snapshot. Later source edits can change them. The existing API key is hidden in the guide only; app source is unchanged.

Start with BeginnerGuide.pdf for the overall search, favorite and preference flows. Use this document when you want to understand a particular file or line.

The file list below and PDF bookmarks help navigation. Some platform/Room/browser setup goes beyond the modules' short examples. Generated files, binary assets and Xcode project internals are outside the handwritten-source scope.

This volume: 39 files; 1136 annotated source lines.

Files in reading order

1. androidApp/build.gradle.kts
2. androidApp/src/main/AndroidManifest.xml
3. androidApp/src/main/kotlin/com/example/dictionary/MainActivity.kt
4. androidApp/src/main/res/values/strings.xml
5. browserStorage/build.gradle.kts
6. browserStorage/src/commonMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt
7. browserStorage/src/jsMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt
8. browserStorage/src/wasmJsMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt

9. browserStorage/worker/package.json
10. browserStorage/worker/worker.js
11. build.gradle.kts
12. desktopApp/build.gradle.kts
13. desktopApp/src/main/kotlin/com/example/dictionary/main.kt
14. desktopApp/src/test/kotlin/com/example/dictionary/NavigationCompatibilityTest.kt
15. gradle.properties
16. gradle/libs.versions.toml
17. iosApp/Configuration/Config.xcconfig
18. iosApp/iosApp/ContentView.swift
19. iosApp/iosApp/Info.plist
20. iosApp/iosApp/iOSApp.swift
21. settings.gradle.kts
22. shared/build.gradle.kts
23. shared/src/androidMain/kotlin/com/example/dictionary/data/AppStorage.kt
24. shared/src/androidMain/kotlin/com/example/dictionary/data/HttpClientFactory.android.kt
25. shared/src/iosMain/kotlin/com/example/dictionary/MainViewController.kt
26. shared/src/iosMain/kotlin/com/example/dictionary/data/AppStorage.kt
27. shared/src/iosMain/kotlin/com/example/dictionary/data/HttpClientFactory.ios.kt
28. shared/src/jsMain/kotlin/com/example/dictionary/data/AppStorage.kt
29. shared/src/jsMain/kotlin/com/example/dictionary/data/HttpClientFactory.js.kt
30. shared/src/jvmMain/kotlin/com/example/dictionary/data/AppStorage.kt
31. shared/src/jvmMain/kotlin/com/example/dictionary/data/HttpClientFactory.jvm.kt
32. shared/src/jvmTest/kotlin/com/example/dictionary/PersistenceTest.kt
33. shared/src/wasmJsMain/kotlin/com/example/dictionary/data/AppStorage.kt
34. shared/src/wasmJsMain/kotlin/com/example/dictionary/data/HttpClientFactory.wasmJs.kt
35. webApp/build.gradle.kts
36. webApp/src/webMain/kotlin/com/example/dictionary/main.kt
37. webApp/src/webMain/resources/index.html
38. webApp/src/webMain/resources/styles.css
39. webApp/webpack.config.d/storage.js

1. build.gradle.kts

androidApp/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.kotlin.gradle.dsl.JvmTarget
```

Makes JvmTarget available here from org.jetbrains.kotlin.gradle.dsl.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
alias(libs.plugins.androidApplication)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
alias(libs.plugins.composeCompiler)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
kotlin {
```

Gradle setup: configures kotlin for the surrounding module/target. This runs during building, not when a user searches.

Line 9

```
compilerOptions {
```

Gradle setup: configures compilerOptions for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
jvmTarget = JvmTarget.JVM_11
```

Gradle setup: configures jvmTarget = JvmTarget.JVM_11 for the surrounding module/target. This runs during building, not when a user searches.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 12

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 13

```
dependencies {
```

Gradle setup: configures dependencies for the surrounding module/target. This runs during building, not when a user searches.

Line 14

```
implementation(project(":shared"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 15

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 16

```
implementation(libs.androidx.activity.compose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 17

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 18

```
implementation(libs.compose.uiToolingPreview)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 19

```
debugImplementation(libs.compose.uiTooling)
```

Gradle setup: configures debugImplementation(libs.compose.uiTooling) for the surrounding module/target. This runs during building, not when a user searches.

Line 20

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
android {
```

Gradle setup: configures android for the surrounding module/target. This runs during building, not when a user searches.

Line 23

```
namespace = "com.example.dictionary"
```

Gradle setup: configures namespace = "com.example.dictionary" for the surrounding module/target. This runs during building, not when a user searches.

Line 24

```
compileSdk =
libs.versions.android.compileSdk.get().toInt()
```

Gradle setup: configures compileSdk = libs.versions.android.compileSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 25

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 26

```
defaultConfig {
```

Gradle setup: configures defaultConfig for the surrounding module/target. This runs during building, not when a user searches.

Line 27

```
applicationId = "com.example.dictionary"
```

Gradle setup: configures applicationId = "com.example.dictionary" for the surrounding module/target. This runs during building, not when a user searches.

Line 28

```
minSdk = libs.versions.android.minSdk.get().toInt()
```

Gradle setup: configures minSdk = libs.versions.android.minSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 29

```
targetSdk = libs.versions.android.targetSdk.get().toInt()
```

Gradle setup: configures targetSdk = libs.versions.android.targetSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 30

```
versionCode = 1
```

Gradle setup: configures versionCode = 1 for the surrounding module/target. This runs during building, not when a user searches.

Line 31

```
versionName = "1.0"
```

Gradle setup: configures versionName = "1.0" for the surrounding module/target. This runs during building, not when a user searches.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 33

```
packaging {
```

Gradle setup: configures packaging for the surrounding module/target. This runs during building, not when a user searches.

Line 34

```
resources {
```

Gradle setup: configures resources for the surrounding module/target. This runs during building, not when a user searches.

Line 35

```
excludes += "/META-INF/{AL2.0,LGPL2.1}"
```

Gradle setup: configures excludes += "/META-INF/{AL2.0,LGPL2.1}" for the surrounding module/target. This runs during building, not when a user searches.

Line 36

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 37

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 38

```
buildTypes {
```

Gradle setup: configures buildTypes for the surrounding module/target. This runs during building, not when a user searches.

Line 39

```
release {
```

Gradle setup: configures release for the surrounding module/target. This runs during building, not when a user searches.

Line 40

```
isMinifyEnabled = false
```

Gradle setup: configures isMinifyEnabled = false for the surrounding module/target. This runs during building, not when a user searches.

Line 41

```
proguardFiles(
```

Gradle setup: configures proguardFiles(for the surrounding module/target. This runs during building, not when a user searches.

Line 42

```
getDefaultProguardFile("proguard-android-optimize.txt"),
```

Gradle setup: configures getDefaultProguardFile("proguard-android-optimize.txt"), for the surrounding module/target. This runs during building, not when a user searches.

Line 43

```
"proguard-rules.pro"
```

Gradle setup: configures "proguard-rules.pro" for the surrounding module/target. This runs during building, not when a user searches.

Line 44

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 45

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 47

```
compileOptions {
```

Gradle setup: configures compileOptions for the surrounding module/target. This runs during building, not when a user searches.

Line 48

```
sourceCompatibility = JavaVersion.VERSION_11
```

Gradle setup: configures sourceCompatibility = JavaVersion.VERSION_11 for the surrounding module/target. This runs during building, not when a user searches.

Line 49

```
targetCompatibility = JavaVersion.VERSION_11
```

Gradle setup: configures targetCompatibility = JavaVersion.VERSION_11 for the surrounding module/target. This runs during building, not when a user searches.

Line 50

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 51

```
buildFeatures {
```

Gradle setup: configures buildFeatures for the surrounding module/target. This runs during building, not when a user searches.

Line 52

```
compose = true
```

Gradle setup: configures compose = true for the surrounding module/target. This runs during building, not when a user searches.

Line 53

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 54

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

2. AndroidManifest.xml

androidApp/src/main/AndroidManifest.xml

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
<?xml version="1.0" encoding="utf-8"?>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
<manifest xmlns:android="http://schemas.android.com/apk/r
es/android">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 3

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 4

```
<uses-permission
android:name="android.permission.INTERNET" />
```

Allows the Android app to make Internet requests.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
<application
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 7

```
android:allowBackup="true"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 8

```
android:icon="@mipmap/ic_launcher"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 9

```
android:label="@string/app_name"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 10

```
android:roundIcon="@mipmap/ic_launcher_round"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 11

```
android:supportsRtl="true"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 12

```
android:theme="@android:style/Theme.Material.Light.NoActi
onBar">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 13

```
<activity
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 14

```
android:exported="true"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 15

```
android:name=".MainActivity">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 16

```
<intent-filter>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 17

```
<action android:name="android.intent.action.MAIN"/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 18

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 19

```
<category
android:name="android.intent.category.LAUNCHER"/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 20

```
</intent-filter>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 21

```
<intent-filter>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 22

```
<action android:name="android.intent.action.VIEW" />
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 23

```
<category android:name="android.intent.category.DEFAULT"
/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 24

```
<category
android:name="android.intent.category.BROWSABLE" />
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 25

```
<data android:scheme="dictionary" android:host="word" />
```

Registers dictionary://word/... as an Android app URL pattern.

Line 26

```
</intent-filter>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 27

```
</activity>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 28

```
</application>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 29

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 30

```
</manifest>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

3. MainActivity.kt

androidApp/src/main/kotlin/com/example/dictionary/MainActivity.kt

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import android.os.Bundle
```

Makes Bundle available here from android.os.

Line 4

```
import com.example.dictionary.data.createAppStorage
```

Makes createAppStorage available here from com.example.dictionary.data.

Line 5

```
import androidx.activity.ComponentActivity
```

Makes ComponentActivity available here from androidx.activity.

Line 6

```
import androidx.activity.compose.setContent
```

Makes setContent available here from androidx.activity.compose.

Line 7

```
import androidx.activity.enableEdgeToEdge
```

Makes enableEdgeToEdge available here from androidx.activity.

Line 8

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 9

```
class MainActivity : ComponentActivity() {
```

Defines MainActivity as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 10

```
override fun onCreate(savedInstanceState: Bundle?) {
```

Defines onCreate(). This fulfills the matching interface/parent operation. Inputs are inside parentheses; the return type describes its answer.

Line 11

```
enableEdgeToEdge()
```

Allows the Android content to draw edge-to-edge; Scaffold padding helps place content around its bars.

Line 12

```
super.onCreate(savedInstanceState)
```

Runs the Activity's parent initialization before showing the app.

Line 13

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 14

```
val storage = createAppStorage(applicationContext)
```

Creates/retains the platform's persistent storage objects before passing them into the shared App.

Line 15

```
setContent {
```

Makes the following Compose UI the Android Activity's content.

Line 16

```
App(storage)
```

Displays the shared Compose App and supplies the storage built by this platform.

Line 17

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 19

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

4. strings.xml

androidApp/src/main/res/values/strings.xml

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
<resources>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
<string name="app_name">dictionary</string>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 3

```
</resources>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

5. build.gradle.kts

browserStorage/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.kotlin.gradle.ExperimentalWasmDsl
```

Makes ExperimentalWasmDsl available here from org.jetbrains.kotlin.gradle.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
alias(libs.plugins.kotlinMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 6

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 7

```
kotlin {
```

Gradle setup: configures kotlin for the surrounding module/target. This runs during building, not when a user searches.

Line 8

```
js {
```

Gradle setup: configures js for the surrounding module/target. This runs during building, not when a user searches.

Line 9

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 12

```
@OptIn(ExperimentalWasmDsl::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 13

```
wasmJs {
```

Gradle setup: configures wasmJs for the surrounding module/target. This runs during building, not when a user searches.

Line 14

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 15

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 16

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 17

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 18

```
sourceSets {
```

Gradle setup: configures sourceSets for the surrounding module/target. This runs during building, not when a user searches.

Line 19

```
commonMain.dependencies {
```

Starts dependencies for commonMain. This determines which targets can see the following libraries.

Line 20

```
api(libs.androidx.sqlite.web)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 21

```
implementation(npm("dictionary-sqlite-worker",
layout.projectDirectory.dir("worker").asFile))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 22

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 23

```
wasmJsMain.dependencies {
```

Starts dependencies for wasmJsMain. This determines which targets can see the following libraries.

Line 24

```
implementation("org.jetbrains.kotlinx:kotlinx-browser:0.5
.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 25

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 26

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 27

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

6. BrowserDriver.kt (commonMain)

browserStorage/src/commonMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
package com.example.dictionary.browser
```

Places this file in the com.example.dictionary.browser namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.sqlite.driver.web.WebWorkerSQLiteDriver
```

Makes WebWorkerSQLiteDriver available here from androidx.sqlite.driver.web.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
// Browser-only infrastructure. App/domain/presentation
code does not depend on JS APIs.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 6

```
expect fun createBrowserDriver(): WebWorkerSQLiteDriver
```

Declares a shared operation that target-specific actual code must implement.

7. BrowserDriver.kt (jsMain)

browserStorage/src/jsMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
package com.example.dictionary.browser
```

Places this file in the com.example.dictionary.browser namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.sqlite.driver.web.WebWorkerSQLiteDriver
```

Makes WebWorkerSQLiteDriver available here from androidx.sqlite.driver.web.

Line 4

```
import org.w3c.dom.Worker
```

Makes Worker available here from org.w3c.dom.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createBrowserDriver(): WebWorkerSQLiteDriver {
```

Supplies this platform’s implementation for the shared expect declaration.

Line 7

```
return WebWorkerSQLiteDriver(Worker(js("""new
URL("dictionary-sqlite-worker/worker.js",
import.meta.url)""")))
```

Connects Room’s browser SQLite driver to a worker that runs database requests outside the UI thread.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

8. BrowserDriver.kt (wasmJsMain)

browserStorage/src/wasmJsMain/kotlin/com/example/dictionary/browser/BrowserDriver.kt

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
package com.example.dictionary.browser
```

Places this file in the com.example.dictionary.browser namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.sqlite.driver.web.WebWorkerSQLiteDriver
```

Makes WebWorkerSQLiteDriver available here from androidx.sqlite.driver.web.

Line 4

```
import org.w3c.dom.Worker
```

Makes Worker available here from org.w3c.dom.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createBrowserDriver(): WebWorkerSQLiteDriver =
 WebWorkerSQLiteDriver(createWorker())
```

Supplies this platform’s implementation for the shared expect declaration.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
@OptIn(ExperimentalWasmJsInterop::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 9

```
private fun createWorker(): Worker =
```

Defines createWorker(). Inputs are inside parentheses; the return type describes its answer.

Line 10

```
js("""new Worker(new
URL("dictionary-sqlite-worker/worker.js",
import.meta.url))""")
```

Browser-only interop: locates the bundled SQLite worker script and creates a worker to handle database requests.

9. package.json

browserStorage/worker/package.json

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
{
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 2

```
"name": "dictionary-sqlite-worker",
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 3

```
"version": "1.0.0",
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 4

```
"private": true,
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 5

```
"license": "Apache-2.0",
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 6

```
"dependencies": { "@sqlite.org/sqlite-wasm":
"3.50.1-build1" }
```

Configuration data: sets this package/resource/schema field. JSON describes data and is not an executable Kotlin statement.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

10. worker.js

browserStorage/worker/worker.js

Additional browser SQLite infrastructure. It connects Kotlin database calls with a JavaScript worker; it is not a PDF example copied verbatim.

Line 1

```
// Adapted from danysantiago/room-web-demo (Apache-2.0);
see ../LICENSE.txt.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 2

```
import sqlite3InitModule from '@sqlite.org/sqlite-wasm';
```

Imports the SQLite WebAssembly initialization function from the worker package dependency.

Line 3

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 4

```
let sqlite3 = null;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
// Maps to track of active database connections and
prepared statements by their unique IDs.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 7

```
const databases = new Map(); // stores databaseId ->
SQLiteDbObject
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 8

```
const statements = new Map(); // stores statementId ->
SQLiteStatementObject
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
// Counters to generate unique IDs for new database
connections and statements.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 11

```
let nextDatabaseId = 0;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 12

```
let nextStatementId = 0;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 13

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 14

```
function openRequest(id, requestData) {
```

Defines a browser-worker helper that processes the request indicated by its name.

Line 15

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 16

```
const newDatabaseId = nextDatabaseId++;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 17

```
const newDatabase = new
sqlite3.oo1.OpfsDb(requestData.fileName);
```

Opens an SQLite database file inside the browser's Origin Private File System.

Line 18

```
databases.set(newDatabaseId, newDatabase);
```

Remembers the open database/statement under its generated numeric ID.

Line 19

```
postMessage({'id': id, data: {'databaseId':
newDatabaseId}});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 20

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 21

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 22

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 23

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 24

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 25

```
function prepareRequest(id, requestData) {
```

Defines a browser-worker helper that processes the request indicated by its name.

Line 26

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 27

```
const newStatementId = nextStatementId++;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 28

```
const resultData = {
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 29

```
'statementId': newStatementId,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 30

```
'parameterCount': 0,
```

Records how many parameter placeholders the prepared SQL expects.

Line 31

```
'columnNames': []
```

Builds the column-name information for the prepared statement reply.

Line 32

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 33

```
const database = databases.get(requestData.databaseId);
```

Finds the database/statement object matching the ID in this request.

Line 34

```
if (!database) {
```

Checks this condition and runs the following branch only when it is true: (! database).

Line 35

```
postMessage({'id': id, error: "Invalid database ID: " +
requestData.databaseId});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 36

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 37

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 38

```
const statement = database.prepare(requestData.sql);
```

Turns the supplied SQL into a prepared statement that can accept parameter values.

Line 39

```
statements.set(newStatementId, statement);
```

Remembers the open database/statement under its generated numeric ID.

Line 40

```
resultData.parameterCount =
sqlite3.capi.sqlite3_bind_parameter_count(statement);
```

Records how many parameter placeholders the prepared SQL expects.

Line 41

```
for (let i = 0; i < statement.columnCount; i++) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 42

```
resultData.columnNames.push(sqlite3.capi.sqlite3_column_n
ame(statement, i));
```

Builds the column-name information for the prepared statement reply.

Line 43

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 44

```
postMessage({'id': id, data: resultData});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 45

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 46

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 47

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 48

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 49

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 50

```
function stepRequest(id, requestData) {
```

Defines a browser-worker helper that processes the request indicated by its name.

Line 51

```
const statement =
statements.get(requestData.statementId);
```

Finds the database/statement object matching the ID in this request.

Line 52

```
if (!statement) {
```

Checks this condition and runs the following branch only when it is true: (! statement).

Line 53

```
postMessage({'id': id, error: "Invalid statement ID: " +
requestData.statementId});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 54

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 55

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 56

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 57

```
const resultData = {
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 58

```
'rows': [],
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 59

```
'columnTypes': []
```

Builds the result’s column-type information so the Kotlin driver can interpret returned values.

Line 60

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 61

```
statement.reset()
```

Resets the prepared statement before executing the next requested set of bindings.

Line 62

```
statement.clearBindings()
```

Clears previous parameter values before binding this request’s values.

Line 63

```
for (let i = 0; i < requestData.bindings.length; i++) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 64

```
statement.bind(i + 1, requestData.bindings[i]);
```

Binds this supplied value to the prepared SQL parameter, using SQLite’s one-based parameter positions.

Line 65

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 66

```
while (statement.step()) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 67

```
if (!resultData.columnTypes.length) {
```

Checks this condition and runs the following branch only when it is true: (! resultData.columnTypes.length).

Line 68

```
for (let i = 0; i < statement.columnCount; i++) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 69

```
resultData.columnTypes.push(sqlite3.capi.sqlite3_column_t
ype(statement, i));
```

Builds the result's column-type information so the Kotlin driver can interpret returned values.

Line 70

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 71

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 72

```
resultData.rows.push(statement.get([]));
```

Copies the current SQL result row into the reply's rows list.

Line 73

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 74

```
postMessage({'id': id, data: resultData});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 75

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 76

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 77

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 78

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 79

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 80

```
function closeRequest(id, requestData) {
```

Defines a browser-worker helper that processes the request indicated by its name.

Line 81

```
if (requestData.statementId != null) {
```

Checks this condition and runs the following branch only when it is true: (requestData.statementId != null).

Line 82

```
const statement =
statements.get(requestData.statementId);
```

Finds the database/statement object matching the ID in this request.

Line 83

```
if (!statement) {
```

Checks this condition and runs the following branch only when it is true: (! statement).

Line 84

```
postMessage({'id': id, error: "Invalid statement ID: " +
requestData.statementId});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 85

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 86

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 87

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 88

```
statement.finalize();
```

Releases the prepared SQL statement’s resources.

Line 89

```
statements.delete(requestData.statementId);
```

Removes the closed resource from the worker’s ID-to-object map.

Line 90

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 91

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 92

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 93

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 94

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 95

```
if (requestData.databaseId != null) {
```

Checks this condition and runs the following branch only when it is true: (requestData.databaseId != null).

Line 96

```
const database = databases.get(requestData.databaseId);
```

Finds the database/statement object matching the ID in this request.

Line 97

```
if (!database) {
```

Checks this condition and runs the following branch only when it is true: (! database).

Line 98

```
postMessage({'id': id, error: "Invalid database ID: " +
requestData.databaseId});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 99

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 100

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 101

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 102

```
database.close();
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 103

```
databases.delete(requestData.databaseId);
```

Removes the closed resource from the worker's ID-to-object map.

Line 104

```
} catch (error) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 105

```
postMessage({'id': id, error: error.message});
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 106

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 107

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 108

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 109

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 110

```
// A map that links command names (strings) to their
respective handler functions.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 111

```
const commandMap = {
```

Maps a request command string to the function that processes it.

Line 112

```
'open': openRequest,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 113

```
'prepare': prepareRequest,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 114

```
'step': stepRequest,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 115

```
'close': closeRequest,
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 116

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 117

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 118

```
function handleMessage(e) {
```

Dispatches this request to the matching open/prepare/step/close handler.

Line 119

```
const requestMsg = e.data;
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 120

```
if (!Object.hasOwn(requestMsg, 'data') || requestMsg.data
 == null) {
```

Checks this condition and runs the following branch only when it is true: (! Object.hasOwn(requestMsg, 'data') || requestMsg.data == null).

Line 121

```
postMessage(
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 122

```
{'id': requestMsg.id, 'error': "Invalid request, missing
'data'."}
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 123

```
);
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 124

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 125

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 126

```
if (!Object.hasOwn(requestMsg.data, 'cmd') ||
requestMsg.data.cmd == null) {
```

Checks this condition and runs the following branch only when it is true: (! Object.hasOwn(requestMsg.data, 'cmd') || requestMsg.data.cmd == null).

Line 127

```
postMessage(
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 128

```
{'id': requestMsg.id, 'error': "Invalid request, missing
'cmd'."}
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 129

```
);
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 130

```
return;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 131

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 132

```
const command = requestMsg.data.cmd;
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 133

```
const requestHandler = commandMap[command];
```

Maps a request command string to the function that processes it.

Line 134

```
if (requestHandler) {
```

Checks this condition and runs the following branch only when it is true: (requestHandler).

Line 135

```
requestHandler(requestMsg.id, requestMsg.data);
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 136

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 137

```
postMessage(
```

Sends a success/error reply back to the Kotlin-side database driver, using the request ID to match the response.

Line 138

```
{'id': requestMsg.id, 'error': "Invalid request, unknown
command: '" + command + "'."}
```

Reads or validates a message from the Kotlin-side driver before using its command/data.

Line 139

```
);
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 140

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 141

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 142

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 143

```
const messageQueue = [];
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 144

```
onmessage = (e) => {
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 145

```
if (!sqlite3) {
```

Checks this condition and runs the following branch only when it is true: (! sqlite3).

Line 146

```
messageQueue.push(e);
```

Queues a request because SQLite has not finished initializing yet.

Line 147

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 148

```
handleMessage(e);
```

Dispatches this request to the matching open/prepare/step/close handler.

Line 149

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 150

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 151

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 152

```
sqlite3InitModule().then(instance => {
```

Initializes SQLite’s WebAssembly module. Queued database requests are handled after initialization succeeds.

Line 153

```
sqlite3 = instance;
```

Worker protocol step: builds, checks or updates this request/reply value inside the current SQL handler.

Line 154

```
while (messageQueue.length > 0) {
```

Repeats the following block for each indicated item, or while the stated condition stays true.

Line 155

```
handleMessage(messageQueue.shift());
```

Dispatches this request to the matching open/prepare/step/close handler.

Line 156

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 157

```
});
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

11. build.gradle.kts

build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 2

```
// this is necessary to avoid the plugins to be loaded
multiple times
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 3

```
// in each subproject's classloader
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 4

```
alias(libs.plugins.androidApplication) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
alias(libs.plugins.androidMultiplatformLibrary) apply
false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
alias(libs.plugins.composeMultiplatform) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 7

```
alias(libs.plugins.composeCompiler) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 8

```
alias(libs.plugins.kotlinJvm) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 9

```
alias(libs.plugins.kotlinMultiplatform) apply false
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

12. build.gradle.kts

desktopApp/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.compose.desktop.application.dsl.Targ
etFormat
```

Makes TargetFormat available here from org.jetbrains.compose.desktop.application.dsl.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
alias(libs.plugins.kotlinJvm)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
alias(libs.plugins.composeMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
alias(libs.plugins.composeCompiler)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 8

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 9

```
dependencies {
```

Gradle setup: configures dependencies for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
implementation(project(":shared"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 11

```
testImplementation(libs.kotlin.testJunit)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 12

```
testImplementation(libs.androidx.navigation.compose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 13

```
testImplementation(libs.androidx.lifecycle.viewmodelCompo
se)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 14

```
testImplementation(libs.androidx.lifecycle.runtimeCompose
)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 15

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 16

```
implementation(compose.desktop.currentOs)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 17

```
implementation(libs.kotlinx.coroutinesSwing)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 18

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 19

```
implementation(libs.compose.uiToolingPreview)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 20

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
compose.desktop {
```

Gradle setup: configures compose.desktop for the surrounding module/target. This runs during building, not when a user searches.

Line 23

```
application {
```

Gradle setup: configures application for the surrounding module/target. This runs during building, not when a user searches.

Line 24

```
mainClass = "com.example.dictionary.MainKt"
```

Gradle setup: configures mainClass = "com.example.dictionary.MainKt" for the surrounding module/target. This runs during building, not when a user searches.

Line 25

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 26

```
nativeDistributions {
```

Gradle setup: configures nativeDistributions for the surrounding module/target. This runs during building, not when a user searches.

Line 27

```
targetFormats(TargetFormat.Dmg, TargetFormat.Msi,
TargetFormat.Deb)
```

Gradle setup: configures targetFormats(TargetFormat.Dmg, TargetFormat.Msi, TargetFormat.Deb) for the surrounding module/target. This runs during building, not when a user searches.

Line 28

```
packageName = "com.example.dictionary"
```

Gradle setup: configures packageName = "com.example.dictionary" for the surrounding module/target. This runs during building, not when a user searches.

Line 29

```
packageVersion = "1.0.0"
```

Gradle setup: configures packageVersion = "1.0.0" for the surrounding module/target. This runs during building, not when a user searches.

Line 30

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 31

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

13. main.kt

desktopApp/src/main/kotlin/com/example/dictionary/main.kt

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.data.createAppStorage
```

Makes createAppStorage available here from com.example.dictionary.data.

Line 4

```
import androidx.compose.runtime.remember
```

Makes remember available here from androidx.compose.runtime.

Line 5

```
import androidx.compose.ui.window.Window
```

Makes Window available here from androidx.compose.ui.window.

Line 6

```
import androidx.compose.ui.window.application
```

Makes application available here from androidx.compose.ui.window.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
fun main() = application {
```

Defines main(). Inputs are inside parentheses; the return type describes its answer.

Line 9

```
val storage = remember { createAppStorage() }
```

Creates/retains the platform's persistent storage objects before passing them into the shared App.

Line 10

```
Window(
```

Creates the desktop application window; following lines configure its title and content.

Line 11

```
onCloseRequest = ::exitApplication,
```

Exits the desktop application when its window is closed.

Line 12

```
title = "dictionary",
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 13

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 14

```
App(storage)
```

Displays the shared Compose App and supplies the storage built by this platform.

Line 15

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 16

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

14. NavigationCompatibilityTest.kt

desktopApp/src/test/kotlin/com/example/dictionary/NavigationCompatibilityTest.kt

Automated checks with assertions. Study after the app logic; this code runs during tests, not normal searches.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.runtime.CompositionLocalProvider
```

Makes CompositionLocalProvider available here from androidx.compose.runtime.

Line 4

```
import androidx.compose.ui.ExperimentalComposeUiApi
```

Makes ExperimentalComposeUiApi available here from androidx.compose.ui.

Line 5

```
import androidx.compose.ui.ImageComposeScene
```

Makes ImageComposeScene available here from androidx.compose.ui.

Line 6

```
import androidx.lifecycle.Lifecycle
```

Makes Lifecycle available here from androidx.lifecycle.

Line 7

```
import androidx.lifecycle.LifecycleOwner
```

Makes LifecycleOwner available here from androidx.lifecycle.

Line 8

```
import androidx.lifecycle.LifecycleRegistry
```

Makes LifecycleRegistry available here from androidx.lifecycle.

Line 9

```
import androidx.lifecycle.ViewModelStore
```

Makes ViewModelStore available here from androidx.lifecycle.

Line 10

```
import androidx.lifecycle.ViewModelStoreOwner
```

Makes ViewModelStoreOwner available here from androidx.lifecycle.

Line 11

```
import androidx.lifecycle.compose.LocalLifecycleOwner
```

Makes LocalLifecycleOwner available here from androidx.lifecycle.compose.

Line 12

```
import androidx.lifecycle.viewmodel.compose.LocalViewMode
lStoreOwner
```

Makes LocalViewModelStoreOwner available here from androidx.lifecycle.viewmodel.compose.

Line 13

```
import androidx.navigation.NavHostController
```

Makes NavHostController available here from androidx.navigation.

Line 14

```
import androidx.navigation.compose.NavHost
```

Makes NavHost available here from androidx.navigation.compose.

Line 15

```
import androidx.navigation.compose.composable
```

Makes composable available here from androidx.navigation.compose.

Line 16

```
import androidx.navigation.compose.rememberNavController
```

Makes rememberNavController available here from androidx.navigation.compose.

Line 17

```
import androidx.savedstate.read
```

Makes read available here from androidx.savedstate.

Line 18

```
import javax.swing.SwingUtilities
```

Makes SwingUtilities available here from javax.swing.

Line 19

```
import kotlin.test.Test
```

Makes Test available here from kotlin.test.

Line 20

```
import kotlin.test.assertEquals
```

Makes assertEquals available here from kotlin.test.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
class NavigationCompatibilityTest {
```

Defines NavigationCompatibilityTest as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 23

```
@OptIn(ExperimentalComposeUiApi::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 24

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 25

```
fun navigationCanRestoreEntriesAndReadWordArguments() {
```

Defines navigationCanRestoreEntriesAndReadWordArguments(). Inputs are inside parentheses; the return type describes its answer.

Line 26

```
SwingUtilities.invokeAndWait {
```

Runs the desktop regression-test body on Swing's UI thread and waits for completion.

Line 27

```
val owner = object : LifecycleOwner, ViewModelStoreOwner
{
```

Stores a value/reference named owner from the expression on the right.

Line 28

```
override val lifecycle =
LifecycleRegistry.createUnsafe(this)
```

Stores a value/reference named lifecycle from the expression on the right.

Line 29

```
override val viewModelStore = ViewModelStore()
```

Stores a value/reference named viewModelStore from the expression on the right.

Line 30

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 31

```
owner.lifecycle.currentState = Lifecycle.State.RESUMED
```

Sets this test owner's lifecycle state so navigation can create and dispose entries correctly.

Line 32

```
var controller: NavHostController? = null
```

Creates a reassignable local value controller from the expression on the right.

Line 33

```
val scene = ImageComposeScene(width = 320, height = 240)
{
```

Stores a value/reference named scene from the expression on the right.

Line 34

```
CompositionLocalProvider(
```

Supplies test environment objects to the Compose navigation content below.

Line 35

```
LocalLifecycleOwner provides owner,
```

Test-only: supplies the lifecycle/state-store owner that a real app window normally provides.

Line 36

```
LocalViewModelStoreOwner provides owner
```

Test-only: supplies the lifecycle/state-store owner that a real app window normally provides.

Line 37

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 38

```
val navController = rememberNavController()
```

Stores a value/reference named navController from the expression on the right.

Line 39

```
controller = navController
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 40

```
NavHost(navController, startDestination = "input") {
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 41

```
composable("input") {}
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 42

```
composable("result/{word}") {}
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 43

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 44

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 45

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 47

```
scene.render().close()
```

Test-only: renders the Compose scene to exercise navigation startup, then releases the produced image.

Line 48

```
val navController = requireNotNull(controller)
```

Stores a value/reference named navController from the expression on the right.

Line 49

```
navController.navigate("result/hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 50

```
scene.render().close()
```

Test-only: renders the Compose scene to exercise navigation startup, then releases the produced image.

Line 51

```
assertEquals("result/{word}",
navController.currentDestination?.route)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 52

```
assertEquals("hello",
navController.currentBackStackEntry?.arguments?.read {
getString("word") })
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 53

```
navController.popBackStack()
```

Test-only: goes back to the previous destination, exercising entry cleanup/restoration.

Line 54

```
scene.render().close()
```

Test-only: renders the Compose scene to exercise navigation startup, then releases the produced image.

Line 55

```
assertEquals("input",
navController.currentDestination?.route)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 56

```
} finally {
```

Runs cleanup whether the attempt succeeded or failed.

Line 57

```
scene.close()
```

Releases the rendered test scene and cancels its Compose effects.

Line 58

```
owner.lifecycle.currentState = Lifecycle.State.DESTROYED
```

Sets this test owner’s lifecycle state so navigation can create and dispose entries correctly.

Line 59

```
owner.viewModelStore.clear()
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 60

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 61

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 62

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 63

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

15. gradle.properties

gradle.properties

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
#Kotlin
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 2

```
kotlin.code.style=official
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 3

```
kotlin.daemon.jvmargs=-Xmx3072M
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 4

```
#Gradle
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 5

```
org.gradle.jvmargs=-Xmx4096M -Dfile.encoding=UTF-8
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 6

```
org.gradle.configuration-cache=true
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 7

```
org.gradle.caching=true
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 8

```
#Android
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 9

```
android.nonTransitiveRClass=true
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

Line 10

```
android.useAndroidX=true
```

Build-tool property: sets the named Gradle/Kotlin/Android configuration value; it is not a per-search operation.

16. libs.versions.toml

gradle/libs.versions.toml

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
[versions]
```

Starts the versions catalog section.

Line 2

```
agp = "9.1.1"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 3

```
android-compileSdk = "37"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 4

```
android-minSdk = "24"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 5

```
android-targetSdk = "37"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 6

```
androidx-activity = "1.13.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 7

```
androidx-appcompat = "1.8.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 8

```
androidx-core = "1.19.1"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 9

```
androidx-espresso = "3.7.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 10

```
androidx-lifecycle = "2.11.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 11

```
androidx-navigation = "2.10.0-rc01"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 12

```
androidx-testExt = "1.3.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 13

```
composeMultiplatform = "1.12.1"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 14

```
junit = "4.13.2"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 15

```
kotlin = "2.4.20"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 16

```
kotlin-wrappers = "2026.9.3"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 17

```
kotlinx-coroutines = "1.11.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 18

```
material3 = "1.12.0-alpha03"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 19

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 20

```
room = "3.0.3"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 21

```
sqlite = "2.7.1"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 22

```
datastore = "1.3.0"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 23

```
ksp = "2.3.12"
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 24

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 25

```
[libraries]
```

Starts the libraries catalog section.

Line 26

```
androidx-navigation-compose = { module =
"org.jetbrains.androidx.navigation:navigation-compose",
version.ref = "androidx-navigation" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 27

```
androidx-room-runtime = { module =
"androidx.room3:room3-runtime", version.ref = "room" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 28

```
androidx-room-compiler = { module =
"androidx.room3:room3-compiler", version.ref = "room" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 29

```
androidx-sqlite-bundled = { module =
"androidx.sqlite:sqlite-bundled", version.ref = "sqlite"
}
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 30

```
androidx-sqlite-web = { module =
"androidx.sqlite:sqlite-web", version.ref = "sqlite" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 31

```
androidx-datastore = { module =
"androidx.datastore:datastore-preferences-core",
version.ref = "datastore" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 32

```
kotlin-test = { module =
"org.jetbrains.kotlin:kotlin-test", version.ref =
"kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 33

```
kotlin-testJunit = { module =
"org.jetbrains.kotlin:kotlin-test-junit", version.ref =
"kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 34

```
junit = { module = "junit:junit", version.ref = "junit" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 35

```
androidx-core-ktx = { module = "androidx.core:core-ktx",
version.ref = "androidx-core" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 36

```
androidx-testExt-junit = { module =
"androidx.test.ext:junit", version.ref =
"androidx-testExt" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 37

```
androidx-espresso-core = { module =
"androidx.test.espresso:espresso-core", version.ref =
"androidx-espresso" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 38

```
androidx-appcompat = { module =
"androidx.appcompat:appcompat", version.ref =
"androidx-appcompat" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 39

```
androidx-activity-compose = { module =
"androidx.activity:activity-compose", version.ref =
"androidx-activity" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 40

```
compose-uiTooling = { module =
"org.jetbrains.compose.ui:ui-tooling", version.ref =
"composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 41

```
androidx-lifecycle-viewmodelCompose = { module = "org.jet
brains.androidx.lifecycle:lifecycle-viewmodel-compose",
version.ref = "androidx-lifecycle" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 42

```
androidx-lifecycle-runtimeCompose = { module = "org.jetbr
ains.androidx.lifecycle:lifecycle-runtime-compose",
version.ref = "androidx-lifecycle" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 43

```
compose-runtime = { module =
"org.jetbrains.compose.runtime:runtime", version.ref =
"composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 44

```
compose-foundation = { module =
"org.jetbrains.compose.foundation:foundation",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 45

```
compose-material3 = { module =
"org.jetbrains.compose.material3:material3", version.ref
= "material3" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 46

```
compose-ui = { module = "org.jetbrains.compose.ui:ui",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 47

```
compose-components-resources = { module =
"org.jetbrains.compose.components:components-resources",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 48

```
compose-uiToolingPreview = { module =
"org.jetbrains.compose.ui:ui-tooling-preview",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 49

```
kotlinx-coroutinesSwing = { module =
"org.jetbrains.kotlinx:kotlinx-coroutines-swing",
version.ref = "kotlinx-coroutines" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 50

```
wrappers-browser = { module =
"org.jetbrains.kotlin-wrappers:kotlin-browser",
version.ref = "kotlin-wrappers" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 51

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 52

```
[plugins]
```

Starts the plugins catalog section.

Line 53

```
androidApplication = { id = "com.android.application",
version.ref = "agp" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 54

```
androidMultiplatformLibrary = { id =
"com.android.kotlin.multiplatform.library", version.ref =
 "agp" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 55

```
composeMultiplatform = { id = "org.jetbrains.compose",
version.ref = "composeMultiplatform" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 56

```
composeCompiler = { id =
"org.jetbrains.kotlin.plugin.compose", version.ref =
"kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 57

```
kotlinJvm = { id = "org.jetbrains.kotlin.jvm",
version.ref = "kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 58

```
kotlinMultiplatform = { id =
"org.jetbrains.kotlin.multiplatform", version.ref =
"kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 59

```
kotlinSerialization = { id =
"org.jetbrains.kotlin.plugin.serialization", version.ref
= "kotlin" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 60

```
ksp = { id = "com.google.devtools.ksp", version.ref =
"ksp" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

Line 61

```
room = { id = "androidx.room3", version.ref = "room" }
```

Version-catalog entry: gives this version, library coordinate or plugin a reusable name for Gradle scripts.

17. Config.xcconfig

iosApp/Configuration/Config.xcconfig

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
TEAM_ID=
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
PRODUCT_NAME=dictionary
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 4

```
PRODUCT_BUNDLE_IDENTIFIER=com.example.dictionary.dictiona
ry$(TEAM_ID)
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
CURRENT_PROJECT_VERSION=1
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 7

```
MARKETING_VERSION=1.0
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

18. ContentView.swift

iosApp/iosApp/ContentView.swift

The native SwiftUI wrapper that opens the shared Kotlin Compose UI on iOS.

Line 1

```
import UIKit
```

Imports UIKit so this Swift wrapper can use its native UI or shared Kotlin declarations.

Line 2

```
import SwiftUI
```

Imports SwiftUI so this Swift wrapper can use its native UI or shared Kotlin declarations.

Line 3

```
import Shared
```

Imports Shared so this Swift wrapper can use its native UI or shared Kotlin declarations.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
struct ComposeView: UIViewControllerRepresentable {
```

Defines ComposeView as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 6

```
func makeUIViewController(context: Self.Context) ->
UIViewController {
```

Defines makeUIViewController(). Inputs are inside parentheses; the return type describes its answer.

Line 7

```
MainViewControllerKt.MainViewController()
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
func updateUIViewController(_ uiViewController:
UIViewController, context: Self.Context) {}
```

Defines updateUIViewController(). Inputs are inside parentheses; the return type describes its answer.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 12

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 13

```
struct ContentView: View {
```

Defines ContentView as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 14

```
var body: some View {
```

Declares the body value and its type for this object or operation. A question mark allows null; a default is used if no value is supplied.

Line 15

```
ComposeView()
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 16

```
.ignoresSafeArea()
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 17

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

19. Info.plist

iosApp/iosApp/Info.plist

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
<?xml version="1.0" encoding="UTF-8"?>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 3

```
<plist version="1.0">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 4

```
<dict>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 5

```
<key>CADisableMinimumFrameDurationOnPhone</key>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 6

```
<true/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 7

```
</dict>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 8

```
</plist>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

20. iOSApp.swift

iosApp/iosApp/iOSApp.swift

The native SwiftUI wrapper that opens the shared Kotlin Compose UI on iOS.

Line 1

```
import SwiftUI
```

Imports SwiftUI so this Swift wrapper can use its native UI or shared Kotlin declarations.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
@main
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 4

```
struct iOSApp: App {
```

Defines iOSApp as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 5

```
var body: some Scene {
```

Declares the body value and its type for this object or operation. A question mark allows null; a default is used if no value is supplied.

Line 6

```
WindowGroup {
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 7

```
ContentView()
```

iOS wrapper setup: hosts the shared Compose controller in the native SwiftUI app. This wrapper does not duplicate the dictionary business logic.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 9

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

21. settings.gradle.kts

settings.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
rootProject.name = "dictionary"
```

Gradle setup: configures rootProject.name = "dictionary" for the surrounding module/target. This runs during building, not when a user searches.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
pluginManagement {
```

Gradle setup: configures pluginManagement for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
repositories {
```

Gradle setup: configures repositories for the surrounding module/target. This runs during building, not when a user searches.

Line 5

```
google {
```

Gradle setup: configures google for the surrounding module/target. This runs during building, not when a user searches.

Line 6

```
mavenContent {
```

Gradle setup: configures mavenContent for the surrounding module/target. This runs during building, not when a user searches.

Line 7

```
includeGroupAndSubgroups("androidx")
```

Gradle setup: configures includeGroupAndSubgroups("androidx") for the surrounding module/target. This runs during building, not when a user searches.

Line 8

```
includeGroupAndSubgroups("com.android")
```

Gradle setup: configures includeGroupAndSubgroups("com.android") for the surrounding module/target. This runs during building, not when a user searches.

Line 9

```
includeGroupAndSubgroups("com.google")
```

Gradle setup: configures includeGroupAndSubgroups("com.google") for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 12

```
mavenCentral()
```

Gradle setup: configures mavenCentral() for the surrounding module/target. This runs during building, not when a user searches.

Line 13

```
gradlePluginPortal()
```

Gradle setup: configures gradlePluginPortal() for the surrounding module/target. This runs during building, not when a user searches.

Line 14

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 15

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 16

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 17

```
dependencyResolutionManagement {
```

Gradle setup: configures dependencyResolutionManagement for the surrounding module/target. This runs during building, not when a user searches.

Line 18

```
repositories {
```

Gradle setup: configures repositories for the surrounding module/target. This runs during building, not when a user searches.

Line 19

```
google {
```

Gradle setup: configures google for the surrounding module/target. This runs during building, not when a user searches.

Line 20

```
mavenContent {
```

Gradle setup: configures mavenContent for the surrounding module/target. This runs during building, not when a user searches.

Line 21

```
includeGroupAndSubgroups("androidx")
```

Gradle setup: configures includeGroupAndSubgroups("androidx") for the surrounding module/target. This runs during building, not when a user searches.

Line 22

```
includeGroupAndSubgroups("com.android")
```

Gradle setup: configures includeGroupAndSubgroups("com.android") for the surrounding module/target. This runs during building, not when a user searches.

Line 23

```
includeGroupAndSubgroups("com.google")
```

Gradle setup: configures includeGroupAndSubgroups("com.google") for the surrounding module/target. This runs during building, not when a user searches.

Line 24

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 25

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 26

```
mavenCentral()
```

Gradle setup: configures mavenCentral() for the surrounding module/target. This runs during building, not when a user searches.

Line 27

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 28

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 29

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 30

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 31

```
id("org.gradle.toolchains.foojay-resolver-convention")
version "1.0.0"
```

Gradle setup: configures id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0" for the surrounding module/target. This runs during building, not when a user searches.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 33

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 34

```
include(":androidApp")
```

Includes these project modules in the Gradle build.

Line 35

```
include(":desktopApp")
```

Includes these project modules in the Gradle build.

Line 36

```
include(":shared")
```

Includes these project modules in the Gradle build.

Line 37

```
include(":webApp")
```

Includes these project modules in the Gradle build.

Line 38

```
include(":browserStorage")
```

Includes these project modules in the Gradle build.

22. build.gradle.kts

shared/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.kotlin.gradle.ExperimentalWasmDsl
```

Makes ExperimentalWasmDsl available here from org.jetbrains.kotlin.gradle.

Line 2

```
import org.jetbrains.kotlin.gradle.dsl.JvmTarget
```

Makes JvmTarget available here from org.jetbrains.kotlin.gradle.dsl.

Line 3

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 4

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 5

```
alias(libs.plugins.kotlinMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
alias(libs.plugins.androidMultiplatformLibrary)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 7

```
alias(libs.plugins.composeMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 8

```
alias(libs.plugins.composeCompiler)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
alias(libs.plugins.kotlinSerialization)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 11

```
alias(libs.plugins.ksp)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 12

```
alias(libs.plugins.room)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 13

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
kotlin {
```

Gradle setup: configures kotlin for the surrounding module/target. This runs during building, not when a user searches.

Line 16

```
listOf(
```

Gradle setup: configures listOf(for the surrounding module/target. This runs during building, not when a user searches.

Line 17

```
iosArm64(),
```

Gradle setup: configures iosArm64(), for the surrounding module/target. This runs during building, not when a user searches.

Line 18

```
iosSimulatorArm64()
```

Gradle setup: configures iosSimulatorArm64() for the surrounding module/target. This runs during building, not when a user searches.

Line 19

```
).forEach { iosTarget ->
```

Gradle setup: configures).forEach { iosTarget -> for the surrounding module/target. This runs during building, not when a user searches.

Line 20

```
iosTarget.binaries.framework {
```

Gradle setup: configures iosTarget.binaries.framework for the surrounding module/target. This runs during building, not when a user searches.

Line 21

```
baseName = "Shared"
```

Gradle setup: configures baseName = "Shared" for the surrounding module/target. This runs during building, not when a user searches.

Line 22

```
isStatic = true
```

Gradle setup: configures isStatic = true for the surrounding module/target. This runs during building, not when a user searches.

Line 23

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 24

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 25

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 26

```
jvm()
```

Gradle setup: configures jvm() for the surrounding module/target. This runs during building, not when a user searches.

Line 27

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 28

```
js {
```

Gradle setup: configures js for the surrounding module/target. This runs during building, not when a user searches.

Line 29

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 30

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 31

```
binaries.executable()
```

Gradle setup: configures binaries.executable() for the surrounding module/target. This runs during building, not when a user searches.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 33

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 34

```
@OptIn(ExperimentalWasmDsl::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 35

```
wasmJs {
```

Gradle setup: configures wasmJs for the surrounding module/target. This runs during building, not when a user searches.

Line 36

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 37

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 38

```
binaries.executable()
```

Gradle setup: configures binaries.executable() for the surrounding module/target. This runs during building, not when a user searches.

Line 39

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 40

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 41

```
android {
```

Gradle setup: configures android for the surrounding module/target. This runs during building, not when a user searches.

Line 42

```
namespace = "com.example.dictionary.shared"
```

Gradle setup: configures namespace = "com.example.dictionary.shared" for the surrounding module/target. This runs during building, not when a user searches.

Line 43

```
compileSdk =
libs.versions.android.compileSdk.get().toInt()
```

Gradle setup: configures compileSdk = libs.versions.android.compileSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 44

```
minSdk = libs.versions.android.minSdk.get().toInt()
```

Gradle setup: configures minSdk = libs.versions.android.minSdk.get().toInt() for the surrounding module/target. This runs during building, not when a user searches.

Line 45

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 46

```
compilerOptions {
```

Gradle setup: configures compilerOptions for the surrounding module/target. This runs during building, not when a user searches.

Line 47

```
jvmTarget = JvmTarget.JVM_11
```

Gradle setup: configures jvmTarget = JvmTarget.JVM_11 for the surrounding module/target. This runs during building, not when a user searches.

Line 48

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 49

```
androidResources {
```

Gradle setup: configures androidResources for the surrounding module/target. This runs during building, not when a user searches.

Line 50

```
enable = true
```

Gradle setup: configures enable = true for the surrounding module/target. This runs during building, not when a user searches.

Line 51

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 52

```
withHostTest {
```

Gradle setup: configures withHostTest for the surrounding module/target. This runs during building, not when a user searches.

Line 53

```
isIncludeAndroidResources = true
```

Gradle setup: configures isIncludeAndroidResources = true for the surrounding module/target. This runs during building, not when a user searches.

Line 54

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 55

```
withDeviceTestBuilder {
```

Gradle setup: configures withDeviceTestBuilder for the surrounding module/target. This runs during building, not when a user searches.

Line 56

```
sourceSetTreeName = "test"
```

Gradle setup: configures sourceSetTreeName = "test" for the surrounding module/target. This runs during building, not when a user searches.

Line 57

```
}.configure {
```

Gradle setup: configures }.configure for the surrounding module/target. This runs during building, not when a user searches.

Line 58

```
instrumentationRunner =
"androidx.test.runner.AndroidJUnitRunner"
```

Gradle setup: configures instrumentationRunner = "androidx.test.runner.AndroidJUnitRunner" for the surrounding module/target. This runs during building, not when a user searches.

Line 59

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 60

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 61

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 62

```
sourceSets {
```

Gradle setup: configures sourceSets for the surrounding module/target. This runs during building, not when a user searches.

Line 63

```
androidMain.dependencies {
```

Starts dependencies for androidMain. This determines which targets can see the following libraries.

Line 64

```
implementation(libs.androidx.sqlite.bundled)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 65

```
implementation("io.ktor:ktor-client-okhttp:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 66

```
implementation(libs.compose.uiToolingPreview)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 67

```
implementation(libs.compose.uiTooling)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 68

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 69

```
commonMain.dependencies {
```

Starts dependencies for commonMain. This determines which targets can see the following libraries.

Line 70

```
implementation(libs.compose.runtime)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 71

```
implementation(libs.compose.foundation)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 72

```
implementation(libs.compose.material3)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 73

```
implementation(libs.compose.ui)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 74

```
implementation(libs.compose.components.resources)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 75

```
implementation(libs.compose.uiToolingPreview)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 76

```
implementation(libs.androidx.lifecycle.viewmodelCompose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 77

```
implementation(libs.androidx.lifecycle.runtimeCompose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 78

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 79

```
// Modules 6 and 9: shared networking and JSON
dependencies.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 80

```
implementation("io.ktor:ktor-client-logging:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 81

```
implementation(libs.androidx.room.runtime)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 82

```
implementation(libs.androidx.datastore)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 83

```
implementation("io.ktor:ktor-client-core:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 84

```
implementation("io.ktor:ktor-client-content-negotiation:3
.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 85

```
implementation("io.ktor:ktor-serialization-kotlinx-json:3
.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 86

```
implementation("org.jetbrains.kotlinx:kotlinx-serializati
on-json:1.7.3")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 87

```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-
core:1.11.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 88

```
implementation(libs.androidx.navigation.compose)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 89

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 90

```
commonTest.dependencies {
```

Starts dependencies for commonTest. This determines which targets can see the following libraries.

Line 91

```
implementation(libs.kotlin.test)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 92

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 93

```
iosMain.dependencies {
```

Starts dependencies for iosMain. This determines which targets can see the following libraries.

Line 94

```
implementation(libs.androidx.sqlite.bundled)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 95

```
implementation("io.ktor:ktor-client-darwin:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 96

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 97

```
jvmTest.dependencies {
```

Starts dependencies for jvmTest. This determines which targets can see the following libraries.

Line 98

```
implementation("io.ktor:ktor-client-mock:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 99

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 100

```
jvmMain.dependencies {
```

Starts dependencies for jvmMain. This determines which targets can see the following libraries.

Line 101

```
implementation(libs.androidx.sqlite.bundled)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 102

```
implementation("io.ktor:ktor-client-cio:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 103

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 104

```
jsMain.dependencies {
```

Starts dependencies for jsMain. This determines which targets can see the following libraries.

Line 105

```
implementation(libs.wrappers.browser)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 106

```
implementation("io.ktor:ktor-client-js:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 107

```
implementation(project(":browserStorage"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 108

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 109

```
wasmJsMain.dependencies {
```

Starts dependencies for wasmJsMain. This determines which targets can see the following libraries.

Line 110

```
implementation("io.ktor:ktor-client-js:3.0.0")
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 111

```
implementation(project(":browserStorage"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 112

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 113

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 114

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 115

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 116

```
dependencies {
```

Gradle setup: configures dependencies for the surrounding module/target. This runs during building, not when a user searches.

Line 117

```
androidRuntimeClasspath(libs.compose.uiTooling)
```

Gradle setup: configures androidRuntimeClasspath(libs.compose.uiTooling) for the surrounding module/target. This runs during building, not when a user searches.

Line 118

```
listOf("kspAndroid", "kspJvm", "kspIosArm64",
"kspIosSimulatorArm64", "kspJs", "kspWasmJs").forEach {
```

Gradle setup: configures listOf("kspAndroid", "kspJvm", "kspIosArm64", "kspIosSimulatorArm64", "kspJs", "kspWasmJs").forEach for the surrounding module/target. This runs during building, not when a user searches.

Line 119

```
add(it, libs.androidx.room.compiler)
```

Adds the Room code generator to the current platform's KSP configuration.

Line 120

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 121

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 123

```
room3 {
```

Gradle setup: configures room3 for the surrounding module/target. This runs during building, not when a user searches.

Line 124

```
schemaDirectory(layout.projectDirectory.dir("schemas"))
```

Records Room's database schema snapshots in this directory for checking/version history.

Line 125

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

23. AppStorage.kt (androidMain)

shared/src/androidMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform's persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import android.content.Context
```

Makes Context available here from android.content.

Line 4

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 5

```
import androidx.datastore.core.FileStorage
```

Makes FileStorage available here from androidx.datastore.core.

Line 6

```
import androidx.datastore.preferences.core.PreferencesFil
eSerializer
```

Makes PreferencesFileSerializer available here from androidx.datastore.preferences.core.

Line 7

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 8

```
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
```

Makes BundledSQLiteDriver available here from androidx.sqlite.driver.bundled.

Line 9

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 10

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 11

```
private var appStorage: AppStorage? = null
```

Retains Android’s shared storage instance for reuse, including Activity recreation.

Line 12

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 13

```
fun createAppStorage(context: Context): AppStorage {
```

Defines createAppStorage(). Inputs are inside parentheses; the return type describes its answer.

Line 14

```
appStorage?.let { return it }
```

If storage already exists, return it instead of opening another DataStore for the same file.

Line 15

```
val appContext = context.applicationContext
```

Uses Android’s application-lifetime context rather than retaining an Activity object.

Line 16

```
val database = Room.databaseBuilder<DictionaryDatabase>(
```

Starts building a Room database using this platform’s file location or browser driver.

Line 17

```
context = appContext,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 18

```
name =
appContext.getDatabasePath("dictionary.db").absolutePath
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 19

```
).setDriver(BundledSQLiteDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 20

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 21

```
storage = FileStorage(
```

Uses JVM/Android file storage for the preference data.

Line 22

```
serializer = PreferencesFileSerializer,
```

Chooses the preference serializer that can read/write DataStore’s preference format.

Line 23

```
produceFile = {
appContext.filesDir.resolve("dictionary.preferences_pb")
}
```

Provides the real file/path where this platform stores preferences.

Line 24

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 25

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 26

```
val storage = AppStorage(database, preferences)
```

Creates/retains the platform's persistent storage objects before passing them into the shared App.

Line 27

```
appStorage = storage
```

Keeps the new Android storage instance so later calls reuse it.

Line 28

```
return storage
```

Sends the following value/expression back as this function's answer: storage.

Line 29

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

24. HttpClientFactory.android.kt (androidMain)

shared/src/androidMain/kotlin/com/example/dictionary/data/HttpClientFactory.android.kt

Creates this platform’s Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.okhttp.OkHttp
```

Makes OkHttp available here from io.ktor.client.engine.okhttp.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform’s implementation for the shared expect declaration.

Line 7

```
return HttpClient(OkHttp) { configureDictionaryClient() }
```

Creates this platform’s HTTP client using OkHttp and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

25. MainViewController.kt (iosMain)

shared/src/iosMain/kotlin/com/example/dictionary/MainViewController.kt

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.ui.window.ComposeUIViewController
```

Makes ComposeUIViewController available here from androidx.compose.ui.window.

Line 4

```
import com.example.dictionary.data.createAppStorage
```

Makes createAppStorage available here from com.example.dictionary.data.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
private val storage by lazy { createAppStorage() }
```

Creates/retains the platform’s persistent storage objects before passing them into the shared App.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
fun MainViewController() = ComposeUIViewController {
App(storage) }
```

Displays the shared Compose App and supplies the storage built by this platform.

26. AppStorage.kt (iosMain)

shared/src/iosMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform's persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.okio.OkioStorage
```

Makes OkioStorage available here from androidx.datastore.core.okio.

Line 5

```
import
androidx.datastore.preferences.core.PreferencesSerializer
```

Makes PreferencesSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
```

Makes BundledSQLiteDriver available here from androidx.sqlite.driver.bundled.

Line 8

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 9

```
import kotlinx.cinterop.ExperimentalForeignApi
```

Makes ExperimentalForeignApi available here from kotlinx.cinterop.

Line 10

```
import okio.FileSystem
```

Makes FileSystem available here from okio.

Line 11

```
import okio.Path.Companion.toPath
```

Makes toPath available here from okio.Path.Companion.

Line 12

```
import platform.Foundation.NSDocumentDirectory
```

Makes NSDocumentDirectory available here from platform.Foundation.

Line 13

```
import platform.Foundation.NSFileManager
```

Makes NSFileManager available here from platform.Foundation.

Line 14

```
import platform.Foundation.NSUserDomainMask
```

Makes NSUserDomainMask available here from platform.Foundation.

Line 15

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 16

```
@OptIn(ExperimentalForeignApi::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 17

```
fun createAppStorage(): AppStorage {
```

Defines createAppStorage(). Inputs are inside parentheses; the return type describes its answer.

Line 18

```
val directory =
NSFileManager.defaultManager.URLForDirectory(
```

Asks iOS for the platform directory used to keep the app’s documents.

Line 19

```
NSDocumentDirectory, NSUserDomainMask, null, true, null
```

Selects the iOS Documents directory, in the user domain, creating it if needed.

Line 20

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
val path = requireNotNull(directory?.path)
```

Requires a valid storage directory path before constructing the database/preferences.

Line 22

```
val database =
Room.databaseBuilder<DictionaryDatabase>(name =
"$path/dictionary.db")
```

Starts building a Room database using this platform’s file location or browser driver.

Line 23

```
.setDriver(BundledSQLiteDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 24

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 25

```
storage = OkioStorage(
```

Uses the cross-platform file interface for iOS preference storage.

Line 26

```
fileSystem = FileSystem.SYSTEM,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 27

```
serializer = PreferencesSerializer,
```

Chooses the preference serializer that can read/write DataStore's preference format.

Line 28

```
producePath = {
"$path/dictionary.preferences_pb".toPath() }
```

Provides the real file/path where this platform stores preferences.

Line 29

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 30

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 31

```
return AppStorage(database, preferences)
```

Returns the database and DataStore together so the platform entry point can pass them to App.

Line 32

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

27. HttpClientFactory.ios.kt (iosMain)

shared/src/iosMain/kotlin/com/example/dictionary/data/HttpClientFactory.ios.kt

Creates this platform’s Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.darwin.Darwin
```

Makes Darwin available here from io.ktor.client.engine.darwin.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform’s implementation for the shared expect declaration.

Line 7

```
return HttpClient(Darwin) { configureDictionaryClient() }
```

Creates this platform’s HTTP client using Darwin and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

28. AppStorage.kt (jsMain)

shared/src/jsMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform's persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.okio.WebLocalStorage
```

Makes WebLocalStorage available here from androidx.datastore.core.okio.

Line 5

```
import
androidx.datastore.preferences.core.PreferencesSerializer
```

Makes PreferencesSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 8

```
import com.example.dictionary.browser.createBrowserDriver
```

Makes createBrowserDriver available here from com.example.dictionary.browser.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
fun createAppStorage(): AppStorage {
```

Defines createAppStorage(). Inputs are inside parentheses; the return type describes its answer.

Line 11

```
val database =
Room.databaseBuilder<DictionaryDatabase>(name =
"dictionary.db")
```

Starts building a Room database using this platform’s file location or browser driver.

Line 12

```
.setDriver(createBrowserDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 13

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 14

```
storage = WebLocalStorage(
```

Uses the browser’s localStorage for the small preference data.

Line 15

```
serializer = PreferencesSerializer,
```

Chooses the preference serializer that can read/write DataStore’s preference format.

Line 16

```
name = "dictionary.preferences"
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 17

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 19

```
return AppStorage(database, preferences)
```

Returns the database and DataStore together so the platform entry point can pass them to App.

Line 20

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

29. HttpClientFactory.js.kt (jsMain)

shared/src/jsMain/kotlin/com/example/dictionary/data/HttpClientFactory.js.kt

Creates this platform's Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.js.Js
```

Makes Js available here from io.ktor.client.engine.js.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform's implementation for the shared expect declaration.

Line 7

```
return HttpClient(Js) { configureDictionaryClient() }
```

Creates this platform's HTTP client using Js and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

30. AppStorage.kt (jvmMain)

shared/src/jvmMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform's persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.FileStorage
```

Makes FileStorage available here from androidx.datastore.core.

Line 5

```
import androidx.datastore.preferences.core.PreferencesFil
eSerializer
```

Makes PreferencesFileSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
```

Makes BundledSQLiteDriver available here from androidx.sqlite.driver.bundled.

Line 8

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 9

```
import java.io.File
```

Makes File available here from java.io.

Line 10

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 11

```
fun createAppStorage(directory: File =
File(System.getProperty("user.home"),
".kmp-dictionary")): AppStorage {
```

Defaults desktop storage to the current user's home folder under .kmp-dictionary.

Line 12

```
directory.mkdirs()
```

Creates the desktop storage directory if it does not already exist.

Line 13

```
val database = Room.databaseBuilder<DictionaryDatabase>(
```

Starts building a Room database using this platform's file location or browser driver.

Line 14

```
name = File(directory, "dictionary.db").absolutePath
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 15

```
).setDriver(BundledSQLiteDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 16

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 17

```
storage = FileStorage(
```

Uses JVM/Android file storage for the preference data.

Line 18

```
serializer = PreferencesFileSerializer,
```

Chooses the preference serializer that can read/write DataStore's preference format.

Line 19

```
produceFile = { File(directory,
"dictionary.preferences_pb") }
```

Provides the real file/path where this platform stores preferences.

Line 20

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 22

```
return AppStorage(database, preferences)
```

Returns the database and DataStore together so the platform entry point can pass them to App.

Line 23

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

31. HttpClientFactory.jvm.kt (jvmMain)

shared/src/jvmMain/kotlin/com/example/dictionary/data/HttpClientFactory.jvm.kt

Creates this platform's Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.cio.CIO
```

Makes CIO available here from io.ktor.client.engine.cio.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform's implementation for the shared expect declaration.

Line 7

```
return HttpClient(CIO) { configureDictionaryClient() }
```

Creates this platform's HTTP client using CIO and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

32. PersistenceTest.kt (jvmTest)

shared/src/jvmTest/kotlin/com/example/dictionary/PersistenceTest.kt

Automated checks with assertions. Study after the app logic; this code runs during tests, not normal searches.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.FileStorage
```

Makes FileStorage available here from androidx.datastore.core.

Line 5

```
import androidx.datastore.preferences.core.PreferencesFil
eSerializer
```

Makes PreferencesFileSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
```

Makes BundledSQLiteDriver available here from androidx.sqlite.driver.bundled.

Line 8

```
import com.example.dictionary.data.DictionaryPreferences
```

Makes DictionaryPreferences available here from com.example.dictionary.data.

Line 9

```
import
com.example.dictionary.data.DictionaryRepositoryImpl
```

Makes DictionaryRepositoryImpl available here from com.example.dictionary.data.

Line 10

```
import
com.example.dictionary.data.configureDictionaryClient
```

Makes configureDictionaryClient available here from com.example.dictionary.data.

Line 11

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 12

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 13

```
import io.ktor.client.engine.mock.MockEngine
```

Makes MockEngine available here from io.ktor.client.engine.mock.

Line 14

```
import io.ktor.client.engine.mock.respond
```

Makes respond available here from io.ktor.client.engine.mock.

Line 15

```
import io.ktor.http.HttpHeaders
```

Makes HttpHeaders available here from io.ktor.http.

Line 16

```
import io.ktor.http.HttpStatusCode
```

Makes HttpStatusCode available here from io.ktor.http.

Line 17

```
import io.ktor.http.headersOf
```

Makes headersOf available here from io.ktor.http.

Line 18

```
import java.io.File
```

Makes File available here from java.io.

Line 19

```
import java.nio.file.Files
```

Makes Files available here from java.nio.file.

Line 20

```
import kotlinx.coroutines.CoroutineScope
```

Makes CoroutineScope available here from kotlinx.coroutines.

Line 21

```
import kotlinx.coroutines.Dispatchers
```

Makes Dispatchers available here from kotlinx.coroutines.

Line 22

```
import kotlinx.coroutines.SupervisorJob
```

Makes SupervisorJob available here from kotlinx.coroutines.

Line 23

```
import kotlinx.coroutines.cancelAndJoin
```

Makes cancelAndJoin available here from kotlinx.coroutines.

Line 24

```
import kotlinx.coroutines.flow.first
```

Makes first available here from kotlinx.coroutines.flow.

Line 25

```
import kotlinx.coroutines.runBlocking
```

Makes runBlocking available here from kotlinx.coroutines.

Line 26

```
import kotlinx.coroutines.withTimeout
```

Makes withTimeout available here from kotlinx.coroutines.

Line 27

```
import kotlin.test.AfterTest
```

Makes AfterTest available here from kotlin.test.

Line 28

```
import kotlin.test.Test
```

Makes Test available here from kotlin.test.

Line 29

```
import kotlin.test.assertEquals
```

Makes assertEquals available here from kotlin.test.

Line 30

```
import kotlin.test.assertFalse
```

Makes assertFalse available here from kotlin.test.

Line 31

```
import kotlin.test.assertNotNull
```

Makes assertNotNull available here from kotlin.test.

Line 32

```
import kotlin.test.assertNull
```

Makes assertNull available here from kotlin.test.

Line 33

```
import kotlin.test.assertTrue
```

Makes assertTrue available here from kotlin.test.

Line 34

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 35

```
class PersistenceTest {
```

Defines PersistenceTest as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 36

```
private val directory =
Files.createTempDirectory("dictionary-test-").toFile()
```

Stores a value/reference named directory from the expression on the right.

Line 37

```
private var database: DictionaryDatabase? = null
```

Creates a reassignable local value database from the expression on the right.

Line 38

```
private val clients = mutableListOf<HttpClient>()
```

Stores a value/reference named clients from the expression on the right. An empty collection will be filled later.

Line 39

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 40

```
private fun openDatabase(): DictionaryDatabase {
```

Defines openDatabase(). Inputs are inside parentheses; the return type describes its answer.

Line 41

```
return Room.databaseBuilder<DictionaryDatabase>(
```

Starts building a Room database using this platform’s file location or browser driver.

Line 42

```
name = File(directory, "test.db").absolutePath
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 43

```
).setDriver(BundledSQLiteDriver()).build().also {
database = it }
```

Supplies the SQLite implementation, then builds the Room database object.

Line 44

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 45

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 46

```
private fun client(
```

Defines client(). Inputs are inside parentheses; the return type describes its answer.

Line 47

```
formal: String = """[{"fl":"noun","shortdef":["a
greeting"],"extra":"ignored"}]""",
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 48

```
urban: String =
"""{"list":[{"word":"hello","definition":"a
[greeting]","example":"hello there"}]}""",
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 49

```
status: HttpStatusCode = HttpStatusCode.OK,
```

Test-helper input: choose the mocked HTTP response status, defaulting to 200 OK.

Line 50

```
onRequest: () -> Unit = {}
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 51

```
): HttpClient {
```

Finishes the test helper inputs and specifies that the function returns a configured HttpClient.

Line 52

```
return HttpClient(MockEngine { request ->
```

Creates a test HTTP client using the mocked engine below. Its responses are supplied by the test, not real online dictionary services.

Line 53

```
onRequest()
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 54

```
val body = if (request.url.host ==
"api.urbandictionary.com") urban else formal
```

Stores a value/reference named body from the expression on the right.

Line 55

```
respond(body, status, headersOf(HttpHeaders.ContentType,
"application/json"))
```

Returns this fake HTTP body/status/headers from the mocked engine.

Line 56

```
}) { configureDictionaryClient() }.also { clients.add(it)
 }
```

Closes the mock handler, applies the real shared client plugins, and retains this client in the test cleanup list.

Line 57

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 58

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 59

```
@AfterTest
```

Runs the following cleanup after each test so separate tests do not share resources/results.

Line 60

```
fun cleanUp() {
```

Defines cleanUp(). Inputs are inside parentheses; the return type describes its answer.

Line 61

```
clients.forEach { it.close() }
```

Releases the object's resources when finished.

Line 62

```
database?.close()
```

Closes the database used by this test so it can be reopened or cleaned up.

Line 63

```
directory.deleteRecursively()
```

Test cleanup: deletes the temporary test directory, not the user's normal app database.

Line 64

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 65

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 66

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 67

```
fun
definitionsAndFavoritesSurviveReopeningWithoutNetwork() =
 runBlocking {
```

Defines definitionsAndFavoritesSurviveReopeningWithoutNetwork(). Inputs are inside parentheses; the return type describes its answer.

Line 68

```
withTimeout(20000) {
```

Limits this test block's running time so a hung operation fails instead of waiting indefinitely.

Line 69

```
val db = openDatabase()
```

Stores a value/reference named db from the expression on the right.

Line 70

```
val repository =
DictionaryRepositoryImpl(db.historyDao(), client(),
"test-key")
```

Receives/keeps repository as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 71

```
val result = assertNotNull(repository.fetchDefinitions("
Hello "))
```

Test check: requires an actual value rather than null; returns that value for further checks when successful.

Line 72

```
assertEquals("a greeting",
result.formalEntries.first().definition)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 73

```
assertEquals("a greeting",
result.urbanEntries.first().definition)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 74

```
repository.toggleFavorite("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 75

```
db.close()
```

Closes the database used by this test so it can be reopened or cleaned up.

Line 76

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 77

```
val reopened = openDatabase()
```

Stores a value/reference named reopened from the expression on the right.

Line 78

```
var requests = 0
```

Creates a reassignable local value requests from the expression on the right.

Line 79

```
val offline = client(status =
HttpStatusCode.ServiceUnavailable) { requests++ }
```

Stores a value/reference named offline from the expression on the right.

Line 80

```
val restored =
DictionaryRepositoryImpl(reopened.historyDao(), offline,
"test-key")
```

Stores a value/reference named restored from the expression on the right.

Line 81

```
assertTrue(assertNotNull(restored.fetchDefinitions("hello
")).isCached)
```

Test check: requires an actual value rather than null; returns that value for further checks when successful.

Line 82

```
assertEquals(0, requests)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 83

```
val history = restored.observeHistory().first()
```

Reads the first/current emitted value here, rather than collecting changes forever.

Line 84

```
assertEquals(1, history.size)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 85

```
assertTrue(history.first().isFavorite)
```

Test check: requires this condition to be true.

Line 86

```
assertEquals("hello", history.first().word)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 87

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 88

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 89

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 90

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 91

```
fun failedRefreshKeepsTheSavedWordAndFavorite() =
runBlocking {
```

Defines failedRefreshKeepsTheSavedWordAndFavorite(). Inputs are inside parentheses; the return type describes its answer.

Line 92

```
withTimeout(20000) {
```

Limits this test block’s running time so a hung operation fails instead of waiting indefinitely.

Line 93

```
val db = openDatabase()
```

Stores a value/reference named db from the expression on the right.

Line 94

```
val original = DictionaryRepositoryImpl(db.historyDao(),
client(), "test-key")
```

Stores a value/reference named original from the expression on the right.

Line 95

```
original.fetchDefinitions("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 96

```
original.toggleFavorite("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 97

```
val offline = DictionaryRepositoryImpl(
```

Stores a value/reference named offline from the expression on the right.

Line 98

```
db.historyDao(), client(status =
HttpStatusCode.ServiceUnavailable), "test-key"
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 99

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 100

```
assertNull(offline.fetchDefinitions("hello", forceRefresh
 = true))
```

Test check: requires the operation’s answer to be null.

Line 101

```
assertEquals("a greeting", assertNotNull(offline.fetchDef
initions("hello")).formalEntries.first().definition)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 102

```
assertTrue(offline.observeHistory().first().first().isFav
orite)
```

Test check: requires this condition to be true.

Line 103

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 104

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 105

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 106

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 107

```
fun successfulRefreshUpdatesDefinitionsWithoutClearingFav
orites() = runBlocking {
```

Defines successfulRefreshUpdatesDefinitionsWithoutClearingFavorites(). Inputs are inside parentheses; the return type describes its answer.

Line 108

```
withTimeout(20000) {
```

Limits this test block’s running time so a hung operation fails instead of waiting indefinitely.

Line 109

```
val db = openDatabase()
```

Stores a value/reference named db from the expression on the right.

Line 110

```
val original = DictionaryRepositoryImpl(db.historyDao(),
client(), "test-key")
```

Stores a value/reference named original from the expression on the right.

Line 111

```
original.fetchDefinitions("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 112

```
original.toggleFavorite("hello")
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 113

```
val updated = DictionaryRepositoryImpl(db.historyDao(),
client(
```

Stores a value/reference named updated from the expression on the right.

Line 114

```
formal = """[{"fl":"noun","shortdef":["updated
greeting"]}]"""
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 115

```
), "test-key")
```

Finishes constructing the test repository with an intentionally fake API key; no live service is contacted.

Line 116

```
val result =
assertNotNull(updated.fetchDefinitions("hello",
forceRefresh = true))
```

Test check: requires an actual value rather than null; returns that value for further checks when successful.

Line 117

```
assertFalse(result.isCached)
```

Test check: requires this condition to be false.

Line 118

```
assertEquals("updated greeting",
result.formalEntries.first().definition)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 119

```
assertTrue(updated.observeHistory().first().first().isFav
orite)
```

Test check: requires this condition to be true.

Line 120

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 121

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 123

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 124

```
fun spellingOnlyAndMalformedResponsesAreNotSavedAsDefinit
ions() = runBlocking {
```

Defines spellingOnlyAndMalformedResponsesAreNotSavedAsDefinitions(). Inputs are inside parentheses; the return type describes its answer.

Line 125

```
withTimeout(20000) {
```

Limits this test block’s running time so a hung operation fails instead of waiting indefinitely.

Line 126

```
val db = openDatabase()
```

Stores a value/reference named db from the expression on the right.

Line 127

```
val spelling = DictionaryRepositoryImpl(db.historyDao(),
client(
```

Stores a value/reference named spelling from the expression on the right.

Line 128

```
formal = """["hello"]""", urban = """{"list":[]}"""
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 129

```
), "test-key")
```

Finishes constructing the test repository with an intentionally fake API key; no live service is contacted.

Line 130

```
assertEquals("hello", assertNotNull(spelling.fetchDefinit
ions("helo")).didYouMeanSuggestion)
```

Test check: compares the expected value (first input) with the actual value (second input); a mismatch fails the test.

Line 131

```
assertTrue(spelling.observeHistory().first().isEmpty())
```

Test check: requires this condition to be true.

Line 132

```
val broken = DictionaryRepositoryImpl(db.historyDao(),
client(
```

Stores a value/reference named broken from the expression on the right.

Line 133

```
formal = "invalid JSON", urban = "invalid JSON"
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 134

```
), "test-key")
```

Finishes constructing the test repository with an intentionally fake API key; no live service is contacted.

Line 135

```
assertNull(broken.fetchDefinitions("hello"))
```

Test check: requires the operation’s answer to be null.

Line 136

```
assertTrue(broken.observeHistory().first().isEmpty())
```

Test check: requires this condition to be true.

Line 137

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 138

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 139

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 140

```
@Test
```

Marks the next function as a test to execute through the test runner.

Line 141

```
fun preferenceSurvivesRecreatingDataStore() = runBlocking
 {
```

Defines preferenceSurvivesRecreatingDataStore(). Inputs are inside parentheses; the return type describes its answer.

Line 142

```
withTimeout(20000) {
```

Limits this test block's running time so a hung operation fails instead of waiting indefinitely.

Line 143

```
val file = File(directory, "test.preferences_pb")
```

Stores a value/reference named file from the expression on the right.

Line 144

```
val firstJob = SupervisorJob()
```

Stores a value/reference named firstJob from the expression on the right.

Line 145

```
val firstStore = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 146

```
storage = FileStorage(PreferencesFileSerializer) { file
},
```

Uses JVM/Android file storage for the preference data.

Line 147

```
scope = CoroutineScope(firstJob + Dispatchers.IO)
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 148

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 149

```
val firstPreferences = DictionaryPreferences(firstStore)
```

Stores a value/reference named firstPreferences from the expression on the right.

Line 150

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 151

```
assertFalse(firstPreferences.observeShowSlang().first())
```

Test check: requires this condition to be false.

Line 152

```
firstPreferences.saveShowSlang(true)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 153

```
} finally {
```

Runs cleanup whether the attempt succeeded or failed.

Line 154

```
firstJob.cancelAndJoin()
```

Stops this test-owned coroutine job and waits for it to end before reopening the same preference file.

Line 155

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 156

```
val secondJob = SupervisorJob()
```

Stores a value/reference named secondJob from the expression on the right.

Line 157

```
val secondStore = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 158

```
storage = FileStorage(PreferencesFileSerializer) { file
},
```

Uses JVM/Android file storage for the preference data.

Line 159

```
scope = CoroutineScope(secondJob + Dispatchers.IO)
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 160

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 161

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 162

```
assertTrue(DictionaryPreferences(secondStore).observeShow
Slang().first())
```

Test check: requires this condition to be true.

Line 163

```
} finally {
```

Runs cleanup whether the attempt succeeded or failed.

Line 164

```
secondJob.cancelAndJoin()
```

Stops this test-owned coroutine job and waits for it to end before reopening the same preference file.

Line 165

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 166

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 167

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 168

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

33. AppStorage.kt (wasmJsMain)

shared/src/wasmJsMain/kotlin/com/example/dictionary/data/AppStorage.kt

Chooses this platform's persistent file/storage location and creates Room plus DataStore. Called at platform startup.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStoreFactory
```

Makes DataStoreFactory available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.core.okio.WebLocalStorage
```

Makes WebLocalStorage available here from androidx.datastore.core.okio.

Line 5

```
import
androidx.datastore.preferences.core.PreferencesSerializer
```

Makes PreferencesSerializer available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.room3.Room
```

Makes Room available here from androidx.room3.

Line 7

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 8

```
import com.example.dictionary.browser.createBrowserDriver
```

Makes createBrowserDriver available here from com.example.dictionary.browser.

Line 9

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 10

```
fun createAppStorage(): AppStorage {
```

Defines createAppStorage(). Inputs are inside parentheses; the return type describes its answer.

Line 11

```
val database =
Room.databaseBuilder<DictionaryDatabase>(name =
"dictionary.db")
```

Starts building a Room database using this platform’s file location or browser driver.

Line 12

```
.setDriver(createBrowserDriver()).build()
```

Supplies the SQLite implementation, then builds the Room database object.

Line 13

```
val preferences = DataStoreFactory.create(
```

Creates the DataStore that persists small preferences using the storage setup below.

Line 14

```
storage = WebLocalStorage(
```

Uses the browser’s localStorage for the small preference data.

Line 15

```
serializer = PreferencesSerializer,
```

Chooses the preference serializer that can read/write DataStore’s preference format.

Line 16

```
name = "dictionary.preferences"
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 17

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 19

```
return AppStorage(database, preferences)
```

Returns the database and DataStore together so the platform entry point can pass them to App.

Line 20

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

34. HttpClientFactory.wasmJs.kt (wasmJsMain)

shared/src/wasmJsMain/kotlin/com/example/dictionary/data/HttpClientFactory.wasmJs.kt

Creates this platform's Ktor engine and reuses the shared JSON/logging/timeout configuration.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.engine.js.Js
```

Makes Js available here from io.ktor.client.engine.js.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
actual fun createHttpClient(): HttpClient {
```

Supplies this platform's implementation for the shared expect declaration.

Line 7

```
return HttpClient(Js) { configureDictionaryClient() }
```

Creates this platform's HTTP client using Js and applies the shared configuration.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

35. build.gradle.kts

webApp/build.gradle.kts

Build/compiler/dependency instructions. These configure the app before it runs.

Line 1

```
import org.jetbrains.kotlin.gradle.ExperimentalWasmDsl
```

Makes ExperimentalWasmDsl available here from org.jetbrains.kotlin.gradle.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
plugins {
```

Gradle setup: configures plugins for the surrounding module/target. This runs during building, not when a user searches.

Line 4

```
alias(libs.plugins.kotlinMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 5

```
alias(libs.plugins.composeMultiplatform)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 6

```
alias(libs.plugins.composeCompiler)
```

Enables the named Gradle plugin using the version catalog. Plugins add build/compiler features.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 8

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 9

```
kotlin {
```

Gradle setup: configures kotlin for the surrounding module/target. This runs during building, not when a user searches.

Line 10

```
js {
```

Gradle setup: configures js for the surrounding module/target. This runs during building, not when a user searches.

Line 11

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 12

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 13

```
binaries.executable()
```

Gradle setup: configures binaries.executable() for the surrounding module/target. This runs during building, not when a user searches.

Line 14

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 15

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 16

```
@OptIn(ExperimentalWasmDsl::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 17

```
wasmJs {
```

Gradle setup: configures wasmJs for the surrounding module/target. This runs during building, not when a user searches.

Line 18

```
browser()
```

Gradle setup: configures browser() for the surrounding module/target. This runs during building, not when a user searches.

Line 19

```
useEsModules()
```

Gradle setup: configures useEsModules() for the surrounding module/target. This runs during building, not when a user searches.

Line 20

```
binaries.executable()
```

Gradle setup: configures binaries.executable() for the surrounding module/target. This runs during building, not when a user searches.

Line 21

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 22

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 23

```
sourceSets {
```

Gradle setup: configures sourceSets for the surrounding module/target. This runs during building, not when a user searches.

Line 24

```
commonMain.dependencies {
```

Starts dependencies for commonMain. This determines which targets can see the following libraries.

Line 25

```
implementation(project(":shared"))
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 26

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 27

```
implementation(libs.compose.ui)
```

Build configuration: adds this dependency to the current source set; api also exposes it to consumers, while testImplementation is for tests.

Line 28

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 29

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 30

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

36. main.kt (webMain)

webApp/src/webMain/kotlin/com/example/dictionary/main.kt

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.data.createAppStorage
```

Makes createAppStorage available here from com.example.dictionary.data.

Line 4

```
import androidx.compose.ui.ExperimentalComposeUiApi
```

Makes ExperimentalComposeUiApi available here from androidx.compose.ui.

Line 5

```
import androidx.compose.ui.window.ComposeViewport
```

Makes ComposeViewport available here from androidx.compose.ui.window.

Line 6

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 7

```
@OptIn(ExperimentalComposeUiApi::class)
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 8

```
fun main() {
```

Defines main(). Inputs are inside parentheses; the return type describes its answer.

Line 9

```
val storage = createAppStorage()
```

Creates/retains the platform’s persistent storage objects before passing them into the shared App.

Line 10

```
ComposeViewport {
```

Mounts the shared Compose UI into the browser page.

Line 11

```
App(storage)
```

Displays the shared Compose App and supplies the storage built by this platform.

Line 12

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 13

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

37. index.html (webMain)

webApp/src/webMain/resources/index.html

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
<!DOCTYPE html>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 2

```
<html lang="en">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 3

```
<head>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 4

```
<meta charset="UTF-8">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 5

```
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 6

```
<title>dictionary</title>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 7

```
<link type="text/css" rel="stylesheet" href="styles.css">
```

Loads the page stylesheet.

Line 8

```
</head>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 9

```
<body style="text-align: center; align-content: center">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 10

```
<svg xmlns="http://www.w3.org/2000/svg" width="40"
height="40" viewBox="0 0 50 50" role="presentation">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 11

```
<circle cx="25" cy="25" r="20" stroke="#ccc"
stroke-width="4" fill="none"/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 12

```
<circle cx="25" cy="25" r="20" stroke="#333"
stroke-width="4" fill="none" stroke-linecap="round"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 13

```
stroke-dasharray="90 125">
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 14

```
<animateTransform attributeName="transform" type="rotate"
 from="0 25 25" to="360 25 25" dur="1s"
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 15

```
repeatCount="indefinite"/>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 16

```
</circle>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 17

```
</svg>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 18

```
<script type="application/javascript"
src="webApp.js"></script>
```

Loads the compiled browser application JavaScript bundle.

Line 19

```
</body>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

Line 20

```
</html>
```

Platform/resource markup: declares this element, setting or text for the containing manifest, page, icon or iOS configuration. It is not Kotlin business logic.

38. styles.css (webMain)

webApp/src/webMain/resources/styles.css

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
html, body {
```

Browser layout/build setting: html, body {; sizes the page or configures the browser development environment.

Line 2

```
width: 100%;
```

Browser layout/build setting: width: 100%;; sizes the page or configures the browser development environment.

Line 3

```
height: 100%;
```

Browser layout/build setting: height: 100%;; sizes the page or configures the browser development environment.

Line 4

```
margin: 0;
```

Browser layout/build setting: margin: 0;; sizes the page or configures the browser development environment.

Line 5

```
padding: 0;
```

Browser layout/build setting: padding: 0;; sizes the page or configures the browser development environment.

Line 6

```
overflow: hidden;
```

Browser layout/build setting: overflow: hidden;; sizes the page or configures the browser development environment.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

39. storage.js

webApp/webpack.config.d/storage.js

Platform entry point or supporting configuration/resource. The annotated lines explain its role in starting or presenting the shared app.

Line 1

```
// SQLite OPFS storage needs a secure,
cross-origin-isolated page.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 2

```
config.devServer = config.devServer || {};
```

Browser layout/build setting: config.devServer = config.devServer || {};; sizes the page or configures the browser development environment.

Line 3

```
config.devServer.headers = {
```

Browser layout/build setting: config.devServer.headers = {; sizes the page or configures the browser development environment.

Line 4

```
...(config.devServer.headers || {}),
```

Browser layout/build setting: ...(config.devServer.headers || {}),; sizes the page or configures the browser development environment.

Line 5

```
'Cross-Origin-Opener-Policy': 'same-origin',
```

Browser layout/build setting: 'Cross-Origin-Opener-Policy': 'same-origin',; sizes the page or configures the browser development environment.

Line 6

```
'Cross-Origin-Embedder-Policy': 'require-corp'
```

Browser layout/build setting: 'Cross-Origin-Embedder-Policy': 'require-corp'; sizes the page or configures the browser development environment.

Line 7

```
};
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.